

KØBENHAVNS
UNIVERSITET

# Information flow analysis for eBPF

Software Security: Mini-project 3

Students: Marcus Friis-Hansen & Anders Hoeck & Rasmus Voss
UNI-ID: lns611 & lpz950 & khp836

October 29, 2026

# Contents

**1 Introduction** **3**

**2 Theoretical Background** **4**
2.1 Information Flow Security . . . . . 4
2.2 Explicit and Implicit Flows . . . . . 4
2.3 Flow-Sensitive Analysis . . . . . 5
2.4 Post-dominance frontier . . . . . 5

**3 Implementation** **5**
3.1 Direct information flow . . . . . 5
3.2 Indirect information flow through branches . . . . . 6

**4 Conclusion** **7**

# 1 Introduction

This project focuses on information flow analysis for a reduced version of eBPF. The goal is to track whether secret information can influence public data.[1]

We consider two security levels, High and Low, and describe how these levels can change as a program runs. The main challenge is handling both direct information flow and information flow caused by conditional jumps.

We first explain the basic concepts and then present a proposed analysis for registers, memory, and control flow.

[1] Thomas Jensen, *Information flow analysis*, Software Security, 2026, slides 2–3.

# 2 Theoretical Background

## 2.1 Information Flow Security

The information flow analysis checks whether secret information can influence the public results. We consider two levels of security[2]:

$$\mathcal{L} = \{L, H\}, \qquad L \sqsubseteq H,$$

$L$ means public and therefore *low security*, while $H$ is secret and therefore represents *high security*. The relation is defined as $L \sqsubseteq H$. In this assignment *security context* refers to the security level value $H$ or $L$, for a given instruction.

When values are combined the result is labeled as secret if at least on of the inputs are secret. We express this using the join operator $\sqcup$:

$$L \sqcup L = L, \qquad L \sqcup H = H \sqcup L = H \sqcup H = H.$$

So if we consider the following instructions, where $r_1$ starts as $H$ and $r_2, r_3$ start as $L$:

```
mov r2, 10
mov r3, r1
add r2, r3
mov r3, 20
```

The table shows the security levels after each instruction:

| **Program point** | $r_1$ | $r_2$ | $r_3$ |
|---|---|---|---|
| Initially | $H$ | $L$ | $L$ |
| `mov r2, 10` | $H$ | $L$ | $L$ |
| `mov r3, r1` | $H$ | $L$ | $H$ |
| `add r2, r3` | $H$ | $H$ | $H$ |
| `mov r3, 20` | $H$ | $H$ | $L$ |

The secret information moves from $r_1$ to $r_3$ first and then to $r_2$. When $r_3$ is overwritten with the public constant 20, its level returns to $L$, as long as the assignment is not controlled by a secret condition.

## 2.2 Explicit and Implicit Flows

An *explicit flow* happens when secret information is copied or used in a calculation, as in the previous example.

An *implicit flow* happens when a secret determines which instructions are executed. For example:

**if** $secret = 0$ **then**
  $public \leftarrow 1$
**else**
  $public \leftarrow 2$.

[2]Jensen, *Information flow analysis*, slide 6.

The secret is never copied into the public variable. However, the public result reveals whether the secret was zero. Therefore, assignments controlled by secret conditions must also be treated as potentially secret. [3]

## 2.3 Flow-Sensitive Analysis

A flow-sensitive analysis tracks security levels at different points in a program. This means that a register or memory location can change from $L$ to $H$ and back to $L$.

A flow-insensitive analysis, on the other hand, does not distinguish between different program points when tracking security levels.

This approach is inspired by the flow-sensitive type system of Hunt and Sands.[4]

## 2.4 Post-dominance frontier

In a program, some instructions run no matter what, while other run dependent on the outcome of a branch, such as in the case of an IF-statement. Knowing which dependent instructions run can indirectly leak secrets, even though the variable itself is kept secret. The post-dominancce frontier helps us identify these nodes/instructions. 7

A given node(Program point) $A$ post-dominates a node $B$ if every path from $B$ to the exit goes through $A$. In other words, if instruction $B$ is executed, instruction $A$ is also guaranteed to execute later, as the program must move through $A$ to exit. The immediate post-dominator is the first Node $A$, that post-dominates a given $B$.

The post-dominant frontier for a given node $PDF(A)$, is the set of all nodes that are immediate predecessors to nodes post-dominated by $A$, but are not themselves post-dominated by $A$.[5] These nodes sit at the border of providing the guarantee "If Node $B$ runs, then Node $A$ is guaranteed to run". The post dominance frontier (PDF), is in this context used to identify instructions, who do not *guarantee* another given instruction executes, but given a certain condition can make it execute. These are most often instructions containing conditional statements. In cases where the conditional statement is an instruction who depends on high security information, subsequent branches should also be given a high security context, because knowing whether they run, can indirectly leak sensitive information about the conditional statement.

# 3 Implementation

## 3.1 Direct information flow

The central problem is that a program can copy or calculate with a secret value, causing the result to depend on that secret. If the analysis does not track this dependency, it may incorrectly classify the result as public.

[3] Jensen, *Information flow analysis*, slides 10–11.

[4] S. Hunt and D. Sands, *On Flow-Sensitive Security Types*, POPL, 2006. See also Jensen, *Information flow analysis*, slide 51.

[5] https://pages.cs.wisc.edu/ fischer/cs701.f14/lectures/L6.4up.pdf

consider:

```
mov r0, r1
add r0, r2
```

The first instruction copies the value of r1 into r0. Since r1 is classified as High r0 must also be classified as High. The second instruction adds r2 to r0. Even if r2 is Low, the result remains High because it depends on the previous value of r0.
Without tracking these dependencies, the analysis could incorrectly classify a register as Low even though its value depends on secret information. This would make it possible for subsequent operations to use secret-derived values without the analysis recognizing their origin.

To address this problem, our implementation associates a security level with each register and propagates, these levels through instructions. Constants are classified as Low, while register operands use the security level recorded is the current abstract state. If a register's security level is not known, it is conservatively treated as High.

For a move instruction, the destination receives the join of the source operands's security level and the current security context:

$$R^{\sharp\prime}(dst = \ell(src) \sqcup pc$$

For arithmetic instructions such as addition, subtraction and multiplication, the result depends on both the source operand and the previous value of the destination register. We therefore compute:

$$R^{\sharp\prime}(dst) = R^{\sharp}(dst) \sqcup \ell(src) \sqcup pc$$

The join operation ensures that the result is classified as High whenever any of these dependencies is High. The security context (pc) is included to account for instructions executed under secret-dependent control flow.

The implementation updates only the destination register's security level. Registers that are not modified retain their previous levels. Since the analysis is flow-sensitive, overwriting a High register with a public constant can reduce its level to Low when the security context is Low. This allows the analysis to track changes in information flow throughout the program rather than permanently classifying a register according to its earlier values.

## 3.2 Indirect information flow through branches

The problem here is that tracking only direct flow is insufficient to ensure that secret information remains protected. A secret value can influence the program's control flow, which can in turn influence the values written to registers or memory. This is known as implicit information flow.
Consider the following:

```
if (r1 == 0) {
    r0 = 1;
```

```
} else {
    r0 = 0
}
```

We assume that r1 is classified as High and r0 is initially Low. Although the assignments use only public constants, the final value of r0 reveals whether the secret condition was true or false. An analysis that considers only the operands of each assignment would incorrectly clasify r0 as Low.

To account for implicit flows, the analysis introduces a security representing the sensitivity of the control flow. The context is determined by the security level of the condition contorlling an instruction. If a branch depends on a High value, instructions whose execution depends on that branch must account for this dependency.

For an instruction executed under a security context (pc) the resulting security level of an assigned value is computed by joining the context with the level of the value's operands:

$$pc \sqcup \ell_{\text{operands}}$$

Consequently, assigning a Low constant under a High context produces a High result. The same principle applies to register updates and memory stores.

**Determining the extent of a branch**. In an unstructered language such as eBPF, the end of a conditional branch is not explicitly represented by a structured block. The anlysis therefore constructs a control-flow graph, in which nodes represent program points and edges represent possible control-flow transitions.

To determine which instructions depend on a branch, the implementation computes post-dominators and immediate post-dominators.2.4

The implementation uses this information to compute control dependencies. For each branch, it identifies the nodes whose execution is controlled by that branch. The function **controlContext** then combines the security levels of the conditions controlling a given instruction. This also allows nested branches to contributes to the security context.

Finally, the analysis propagates security states through the CFG until a fixed point is reached. At each program point, the incoming states from its predecessors are joined, ensuring that the result conservatively accounts for the different possible execution paths.

# 4 Conclusion

By enforcing these security properties, our analysis propagates security levels throughout the program. It also identifies instructions influenced by secret-dependent branches and accounts for where control-flow paths converge again. This allows the analysis to stop treating instructions as influenced by a branch once they are no longer control-dependent on it.

To evaluate our implementation, we created a test suite covering direct information flows, memory operations, nested branches, and cases where the influence of a branch ends. These tests help verify that the analysis behaves as expected in the scenarios we considered.

Overall, the implementation handles the tested cases as intended. However, passing these tests does not prove the correctness of the analysis, since untested cases may still reveal errors or limitations in the implementation.